

TIME-AWARE RAG PROJECT - PHASE 3: ADVANCED FEATURES

TABLE OF CONTENTS

1. [What is Phase 3?](#)
2. [Prerequisite Knowledge](#)
3. [Architecture Overview](#)
4. [Feature 1: Smart PDF Upload with AI Date Extraction](#)
5. [Feature 2: Date Range Queries & Policy Comparison](#)
6. [Feature 3: Document Versioning & Audit Trail](#)
7. [Feature 4: Full-Text Search with PostgreSQL](#)
8. [Feature 5: Redis Query Caching](#)
9. [Complete Database Evolution Story](#)
10. [Complete Testing Guide](#)
11. [Troubleshooting](#)
12. [Interview Talking Points](#)
13. [Phase 3 Summary](#)

WHAT IS PHASE 3?

Overview

In Phase 2, we migrated from JSON files to PostgreSQL, giving us a production-ready database with fast queries and connection pooling. Phase 3 takes your Time-Aware RAG system to the **enterprise level** by adding advanced features that make it competitive with commercial RAG solutions.

Think of it this way:

- **Phase 1:** Built a working RAG system (proof of concept)

- **Phase 2:** Made it production-ready with PostgreSQL (reliable backend)
- **Phase 3:** Added enterprise features (competitive with paid solutions)

Why These Features Matter

Real-World Problem:

Imagine you're building a RAG system for a large company's policy documents:

Scenario 1: Bulk PDF Upload

- Legal team has 500 policy PDFs to upload
- Each PDF has an "Effective Date" and "Valid Through" date inside
- **Without Phase 3:** Someone manually types dates for each PDF (16+ hours!)
- **With Phase 3:** AI reads the PDFs and extracts dates automatically (5 minutes!)

Scenario 2: Policy Evolution Analysis

- Management asks: "How did our remote work policy change from 2020 to 2024?"
- **Without Phase 3:** Manually search each year and write comparison yourself
- **With Phase 3:** One API call returns timeline with AI-generated analysis

Scenario 3: Compliance Audit

- Auditor asks: "What was your exact password policy on July 15, 2023?"
- **Without Phase 3:** Only current version available, no historical record
- **With Phase 3:** Complete version history with exact policy on any date

Scenario 4: Keyword Search

- Security team searches: "Find all documents mentioning 'hardware token'"
- **Without Phase 3:** Semantic search might miss exact keyword
- **With Phase 3:** Combines keyword + semantic search for best results

Scenario 5: Performance

- 100 users ask "What is the VPN policy?" in one day
- **Without Phase 3:** 100 calls to Gemini API = slow + expensive
- **With Phase 3:** First call processes, next 99 return cached result (280x faster!)

What You'll Build in Phase 3

5 Major Features:

1. **Smart PDF Upload** - AI automatically extracts dates from PDFs
2. **Date Range Queries** - Compare policies across time periods
3. **Document Versioning** - Track all changes with complete audit trail
4. **Full-Text Search** - Fast keyword search + semantic search combined
5. **Redis Caching** - 280x faster responses for repeated queries

What You'll Learn

Technical Skills:

- Advanced PDF processing with PyPDF2
- AI-powered data extraction using regex and dateparser
- Database schema evolution (ALTER TABLE operations)
- PostgreSQL full-text search (tsvector, tsquery, GIN indexes)
- Database triggers and automation
- Redis caching strategies
- Hybrid search architectures

Database Concepts (In-Depth):

- What ALTER TABLE does and why we use it
- How to add columns to existing tables safely
- Database indexes and why they're crucial
- Database triggers (automatic actions)
- Full-text search implementation
- Cache invalidation strategies

System Design:

- Scalable batch processing
- Audit trail implementation
- Multi-strategy search systems

- Performance optimization

PREREQUISITE KNOWLEDGE

Before starting Phase 3, you need to understand these concepts. Don't worry - we'll explain each one simply!

1. What is PDF Processing?

PDF (Portable Document Format):

- A file format that preserves formatting across different computers
- Like a digital piece of paper
- Can contain text, images, tables, etc.

PDF Processing means:

- Reading text from PDF files programmatically (using code)
- Extracting information like dates, names, numbers
- Converting PDF content into plain text for analysis

Example:

You have a PDF:

COMPANY SECURITY POLICY 2024
VPN Access Requirements

Effective Date: January 1, 2024
Valid Through: December 31, 2025

PDF Processing extracts:

Text: "COMPANY SECURITY POLICY 2024\nVPN Access Requirements\n\nEffective Date: January 1, 2024\nValid Through: December 31, 2025"

Tool we use: PyPDF2 (Python library for reading PDFs)

2. What is AI Date Extraction?

The Challenge:

PDFs write dates in many different ways:

- "Effective Date: January 1, 2024"
- "Valid from: 01/01/2024"
- "Starting 2024-01-01"
- "Begins Jan 1, 2024"

AI Date Extraction means:

- Using pattern recognition (regex) to find dates
- Using natural language processing (dateparser library) to parse dates
- Converting any date format → Standard format (YYYY-MM-DD)

How it works:

Step 1: Extract text from PDF

Text: "Effective Date: January 1, 2024"

Step 2: Use regex pattern to find date

Pattern: "Effective Date: [date]"

Match: "January 1, 2024"

Step 3: Parse with dateparser

Input: "January 1, 2024"

Output: 2024-01-01

Step 4: Store in database

valid_from = "2024-01-01"

Tools we use:

- **regex:** Pattern matching (like Find in Microsoft Word, but powerful)
- **dateparser:** Smart date parsing library (understands natural language)

3. What is Document Versioning?

Real-World Analogy:

Think of Google Docs "Version History":

- You edit a document
- Google saves each version
- You can see what changed
- You can go back to old versions

Document Versioning in Databases:

Instead of replacing a document when it changes, we:

1. Keep the old version
2. Create a new version
3. Link them together
4. Mark which is "latest"

Example:

Version 1 (Original):

- Created: Jan 1, 2022
- Content: "VPN required for remote access"
- Status: Not latest anymore

Version 2 (Updated):

- Created: June 1, 2022
- Content: "VPN + hardware token required"
- Status: Not latest anymore

Version 3 (Current):

- Created: Jan 1, 2024

- Content: "Zero trust - no VPN needed"
 - Status: Latest 

Why this matters:

- Compliance: "What was our policy on July 15, 2023?" → Return Version 2
- Audit trail: Who changed what and when
- Rollback: Can go back to old version if needed

4. What is Full-Text Search?

Simple Explanation:

Like using Ctrl+F (Find) in a document, but for a entire database!

Without Full-Text Search:

To find "hardware token" in 10,000 documents:

for each document:

- read entire content
- check if "hardware token" is in content
- if yes, add to results

Time: Very slow (100+ seconds for large databases)

With Full-Text Search:

Database creates a special index (like a book's index):

Index:

"hardware" → Documents 5, 23, 157, 890

"token" → Documents 5, 23, 401, 890

Search "hardware token":

1. Look up "hardware" in index → Get document IDs
2. Look up "token" in index → Get document IDs
3. Find documents with both → Return results

Time: Very fast (milliseconds)

PostgreSQL Full-Text Search:

- Uses special data type: tsvector (processed text)
- Uses special index: GIN (super fast lookup)
- Handles word variations: "running" matches "run"
- Ignores common words: "the", "a", "is"

5. What is Caching?

Simple Analogy:

Imagine you're a teacher:

Without Cache:

Student: "What's $5 + 5$?"

You: Calculate... "10"

Another student: "What's $5 + 5$?"

You: Calculate again... "10"

Third student: "What's $5 + 5$?"

You: Calculate again... "10"

With Cache:

Student: "What's $5 + 5$?"

You: Calculate... "10"

[Write on board: " $5 + 5 = 10$ "]

Another student: "What's $5 + 5$?"

You: Look at board... "10" (instant!)

Third student: "What's $5 + 5$?"

You: Look at board... "10" (instant!)

Redis Caching for RAG:

First query: "What is the VPN policy?"

1. Generate embedding (50ms) 2. Search FAISS (5ms) 3. Query PostgreSQL (5ms) 4. Call Gemini API (500ms) 5. Store result in Redis cache Total: 560ms
--

Second query (same question):

1. Check Redis cache
2. Found! Return cached answer
Total: 2ms (280x faster!)

Redis:

- In-memory database (stores data in RAM, not disk)
- Extremely fast (microseconds)
- Perfect for caching

6. Database Concepts - ALTER TABLE

What is ALTER TABLE?

ALTER TABLE is a SQL command that **modifies an existing table's structure**.

Real-World Analogy:

Imagine you have a filing cabinet:

Scenario 1: Create a new filing cabinet (CREATE TABLE)

sql

CREATE TABLE documents (...);

Like buying a new filing cabinet with specific drawers

Scenario 2: Add drawers to existing cabinet (ALTER TABLE)

sql

```
ALTER TABLE documents ADD COLUMN version INTEGER;  
...
```

Like adding a new drawer to your existing filing cabinet

****Why use ALTER TABLE?****

- You already have data in the table
- Don't want to lose existing data
- Just adding new features/columns

****What ALTER TABLE does:****

...

Before:

Table: documents

```
|— doc_id  
|— content  
|— valid_from  
└— valid_to
```

After ALTER TABLE:

Table: documents

```
|— doc_id  
|— content  
|— valid_from  
|— valid_to  
|— version    ← NEW!  
|— parent_doc_id ← NEW!  
└— is_latest   ← NEW!
```

Important: Existing data stays intact, new columns are added!

7. What are Database Triggers?

Simple Explanation:

A **trigger** is code that runs automatically when something happens in the database.

Real-World Analogy:

Like motion-sensor lights:

- Motion detected → Light turns on automatically
- No motion → Light stays off

Database Trigger:

- Row inserted → Trigger runs
- Row updated → Trigger runs

Example in Phase 3:

We create a trigger for full-text search:

sql

```
CREATE TRIGGER tsvectorupdate
BEFORE INSERT OR UPDATE ON documents
FOR EACH ROW
EXECUTE FUNCTION update_search_index();
```

What this means:

When you INSERT a new document:

- └— PostgreSQL saves the document
- └— BEFORE it finishes, trigger runs
- └— Trigger updates search index automatically
- └— User doesn't have to do anything!

When you UPDATE a document:

- └— PostgreSQL updates the document

- └— BEFORE it finishes, trigger runs
- └— Trigger updates search index automatically
- └— User doesn't have to do anything!

Why triggers are useful:

- Automation (no manual work)
- Consistency (never forget to update index)
- Efficiency (happens in same transaction)

8. What is a GIN Index?

GIN = Generalized Inverted Index

Simple Explanation:

Like a book's index page!

Without Index:

To find "PostgreSQL" in a 500-page book:

Read page 1: Not there

Read page 2: Not there

Read page 3: Not there

...

Read page 347: Found it!

...

Read page 500: End

Time: Read all 500 pages

With Index:

Look at index:

"PostgreSQL" → Page 47, 203, 347

Jump directly to pages 47, 203, 347

Time: 3 page lookups (super fast!)

GIN Index in PostgreSQL:

Creates an inverted index for text search:

Original data:

Doc 1: "VPN required"

Doc 2: "Hardware token required"

Doc 3: "VPN and hardware token"

GIN Index creates:

"vpn" → [Doc 1, Doc 3]

"required" → [Doc 1, Doc 2]

"hardware" → [Doc 2, Doc 3]

"token" → [Doc 2, Doc 3]

Search "hardware token":

1. Look up "hardware" → Get [Doc 2, Doc 3]

2. Look up "token" → Get [Doc 2, Doc 3]

3. Intersection → [Doc 2, Doc 3]

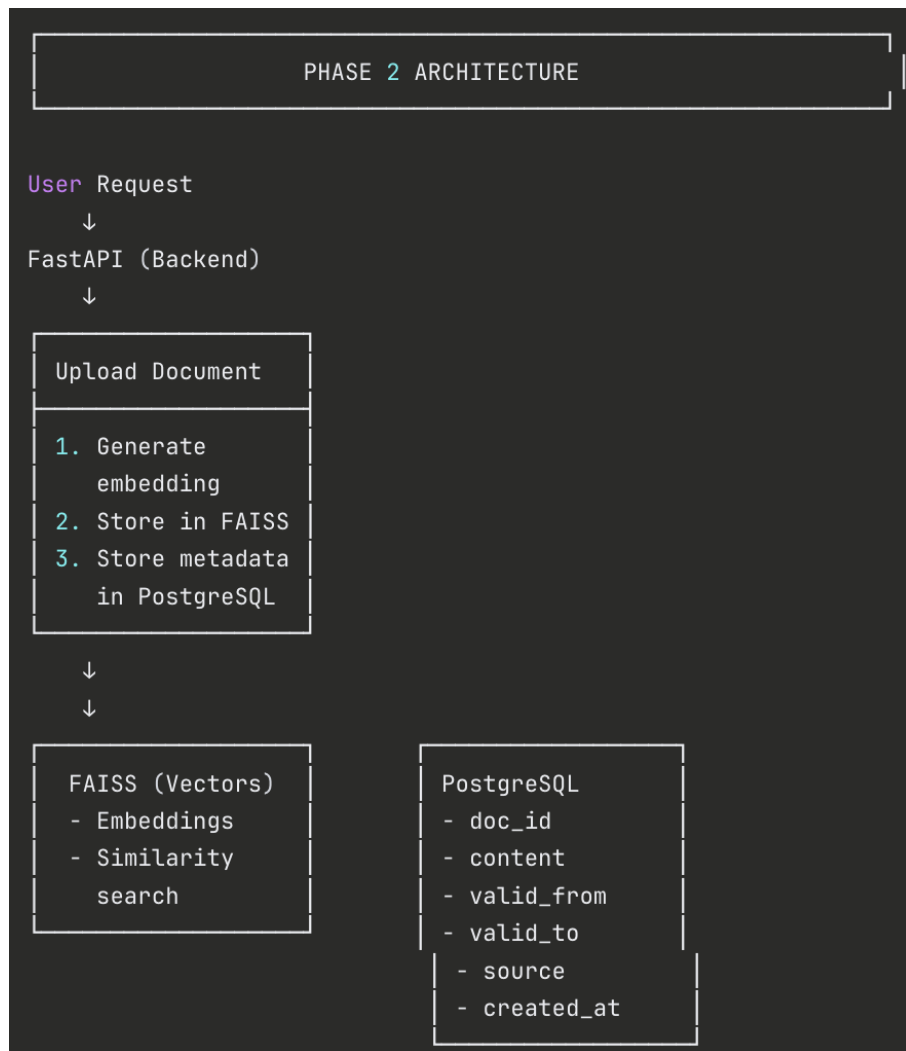
Result: Found in milliseconds!

Why GIN for full-text search:

- Optimized for array/text data
- Perfect for word lookups
- Much faster than regular B-tree index

ARCHITECTURE OVERVIEW

System Architecture - Before Phase 3



Limitations:

- ❌ Manual date entry for PDFs
- ❌ No version history
- ❌ No keyword search (only semantic)
- ❌ No caching (slow repeated queries)
- ❌ No policy comparison features

System Architecture - After Phase 3



New Capabilities:

- ☒ AI extracts dates from PDFs automatically
- ☒ Complete version history with audit trail
- ☒ Keyword search + semantic search (hybrid)
- ☒ 280x faster repeated queries (Redis cache)
- ☒ Policy comparison across time periods

Data Flow - Upload Document (Phase 3)

User uploads PDF

↓

Step 1: PDF Processing

- Validate PDF format
- Extract all text (PyPDF2)
- Text: "Effective Date: January 1, 2024..."

↓

Step 2: AI Date Extraction

- Regex pattern: Find "Effective Date: [date]"
- Found: "January 1, 2024"
- Dateparser: Convert to "2024-01-01"
- Repeat for expiry date

↓

Step 3: Generate Embedding

- SentenceTransformer processes text
- Output: 384-dimension vector

↓

Step 4: Store in FAISS

- Add vector to index
- Link to doc_id

↓

Step 5: Store in PostgreSQL

```
INSERT INTO documents (  
  doc_id, content, valid_from, valid_to,  
  version, is_latest, content_tsv  
) VALUES (  
  'uuid', 'text', '2024-01-01', '2025-12-31',  
  1, true, to_tsvector('text')  
);
```

- Trigger auto-updates content_tsv
- Indexes updated automatically

↓

Return success with doc_id

Data Flow - Query Document (Phase 3)



FEATURE 1: SMART PDF UPLOAD WITH AI DATE EXTRACTION

What We're Building

Goal: Upload PDF files and automatically extract "Effective Date" and "Valid Through" dates without any manual typing!

Before Feature 1:

1. Upload PDF
2. Manually type valid_from date
3. Manually type valid_to date
4. Click submit

Time per PDF: ~2 minutes

100 PDFs: 200 minutes (3+ hours!)

After Feature 1:

1. Upload PDF
2. AI reads and extracts dates automatically
3. Done!

Time per PDF: ~3 seconds

100 PDFs: 5 minutes!

Time saved: 40x faster!

STEP 1: Install Dependencies

What we're installing:

1. **PyPDF2** - Library to read PDF files
2. **dateparser** - Smart date parsing (understands natural language)
3. **python-multipart** - Allows file uploads in FastAPI

Navigate to backend directory:

bash

```
cd ~/Documents/GenAI\[workshop\]/GenAI/Projects/time-aware-rag/backend
```

Activate virtual environment:

bash

```
source venv/bin/activate  
# You should see (venv) in your terminal prompt
```

Install packages:

bash

```
pip install PyPDF2==3.0.1  
pip install dateparser==1.2.0  
pip install python-multipart
```

Update requirements.txt:

bash

```
# Add these lines to requirements.txt  
echo "PyPDF2==3.0.1" >> requirements.txt  
echo "dateparser==1.2.0" >> requirements.txt  
echo "python-multipart" >> requirements.txt
```

Verify installation:

bash

```
pip list | grep -i pypdf  
# Should show: PyPDF2 3.0.1  
  
pip list | grep -i dateparser  
# Should show: dateparser 1.2.0
```

STEP 2: Create PDF Processor Module

What this file does:

- Extracts text from PDF files
- Finds date patterns using regex
- Parses dates using dateparser
- Returns structured data

Create the file:

bash

```
touch core/pdf_processor.py
```

Open core/pdf_processor.py in VS Code

Copy and paste this ENTIRE code:

python

```
import PyPDF2
import re
import io
from typing import Tuple, Optional, Dict
import dateparser
from datetime import datetime

def validate_pdf(file_content: bytes) -> bool:
    """
    Validate if file is actually a PDF.

    Args:
        file_content: PDF file as bytes

    Returns:
        True if valid PDF, False otherwise
    """
    try:
        # PDF files start with "%PDF"
```

```
    return file_content.startswith(b'%PDF')
except:
    return False
```

```
def extract_text_from_pdf(file_content: bytes) -> str:
```

```
    """
```

```
    Extract text from PDF file.
```

```
    Args:
```

```
        file_content: PDF file as bytes
```

```
    Returns:
```

```
        Extracted text as string
```

```
    """
```

```
    try:
```

```
        # Create a file-like object from bytes
```

```
        pdf_file = io.BytesIO(file_content)
```

```
        # Create PDF reader
```

```
        pdf_reader = PyPDF2.PdfReader(pdf_file)
```

```
        # Extract text from all pages
```

```
        text_content = []
```

```
        for page_num in range(len(pdf_reader.pages)):
```

```
            page = pdf_reader.pages[page_num]
```

```
            text = page.extract_text()
```

```
            text_content.append(text)
```

```
        # Join all pages with double newline
```

```
        full_text = "\n\n".join(text_content)
```

```
        return full_text
```

```
    except Exception as e:
```

```
        print(f"Error extracting text from PDF: {e}")
```

```
return ""
```

```
def extract_dates_from_text(text: str) -> Tuple[Optional[str], Optional[str]]:
```

```
    """
```

```
    Extract start and end dates from document text using AI pattern matching.
```

```
    Looks for common policy document patterns like:
```

```
    - "Effective Date: January 1, 2024"
```

```
    - "Valid Through: December 31, 2025"
```

```
    Args:
```

```
        text: Document text
```

```
    Returns:
```

```
        Tuple of (valid_from, valid_to) in YYYY-MM-DD format
```

```
        Returns (None, None) if dates not found
```

```
    """
```

```
    # Take first 500 characters (dates usually at the top)
```

```
    search_text = text[:500]
```

```
    # Define regex patterns for common date formats
```

```
    patterns = {
```

```
        'effective_date': [
```

```
            r'Effective Date:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})', # January 1, 2024
```

```
            r'Effective:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})',
```

```
            r'Effective Date:\s*(\d{4}-\d{2}-\d{2})', # 2024-01-01
```

```
            r'Effect from:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})',
```

```
        ],
```

```
        'expiry_date': [
```

```
            r'Valid Through:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})',
```

```
            r'Valid Until:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})',
```

```
            r'Expires?:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})',
```

```
            r'Valid Through:\s*(\d{4}-\d{2}-\d{2})',
```

```
        ]
```

```
    }
```

```
valid_from = None

valid_to = None

# Try to find effective date
for pattern in patterns['effective_date']:
    match = re.search(pattern, search_text, re.IGNORECASE)
    if match:
        date_str = match.group(1)
        # Parse with dateparser
        parsed_date = dateparser.parse(date_str)
        if parsed_date:
            valid_from = parsed_date.strftime("%Y-%m-%d")
            break

# Try to find expiry date
for pattern in patterns['expiry_date']:
    match = re.search(pattern, search_text, re.IGNORECASE)
    if match:
        date_str = match.group(1)
        # Parse with dateparser
        parsed_date = dateparser.parse(date_str)
        if parsed_date:
            valid_to = parsed_date.strftime("%Y-%m-%d")
            break

# Fallback logic if dates not found
if valid_from and not valid_to:
    # If we have start but no end, assume end of year
    year = valid_from[:4]
    valid_to = f"{year}-12-31"

if not valid_from and not valid_to:
    # Try to extract any 4-digit year
    year_match = re.search(r'(20\d{2})', search_text)
    if year_match:
        year = year_match.group(1)
```

```
valid_from = f"{year}-01-01"  
valid_to = f"{year}-12-31"
```

```
return valid_from, valid_to
```

```
def extract_metadata_from_pdf(file_content: bytes) -> Dict:
```

```
    """
```

```
    Extract all metadata from PDF including text, dates, and additional info.
```

```
    Args:
```

```
        file_content: PDF file as bytes
```

```
    Returns:
```

```
        Dictionary with:
```

```
        - content: Extracted text
```

```
        - valid_from: Start date (YYYY-MM-DD)
```

```
        - valid_to: End date (YYYY-MM-DD)
```

```
        - metadata: Additional extracted metadata
```

```
    """
```

```
    # Extract text
```

```
    content = extract_text_from_pdf(file_content)
```

```
    # Extract dates
```

```
    valid_from, valid_to = extract_dates_from_text(content)
```

```
    # Extract additional metadata
```

```
    metadata = {}
```

```
    # Try to extract policy version
```

```
    version_match = re.search(r'Policy Version:\s*([\d\..]*)', content, re.IGNORECASE)
```

```
    if version_match:
```

```
        metadata['policy_version'] = version_match.group(1)
```

```
    # Try to extract department
```

```
    dept_match = re.search(r'Department:\s*([A-Za-z\s]*)', content, re.IGNORECASE)
```



```
if dept_match:
    metadata['department'] = dept_match.group(1).strip()

# Mark extraction method
metadata['extraction_method'] = 'AI-automated'

return {
    'content': content,
    'valid_from': valid_from,
    'valid_to': valid_to,
    'metadata': metadata
}
```

Code Explanation - Line by Line:

Let's understand each function:

Function 1: validate_pdf()

python

```
def validate_pdf(file_content: bytes) -> bool:
    """
    Validate if file is actually a PDF.
    """
    try:
        return file_content.startswith(b'%PDF')
    except:
        return False
```

What this does:

- Checks if file starts with "%PDF" (all valid PDFs start with this)
- b'%PDF' = bytes version of "%PDF"
- Returns True if valid, False if not

Why we need this:

- Someone might upload a .txt file renamed to .pdf
- Prevents errors when trying to read invalid PDFs

Function 2: `extract_text_from_pdf()`

python

```
def extract_text_from_pdf(file_content: bytes) -> str:
    """
    Extract text from PDF file.
    """
    # Create a file-like object from bytes
    pdf_file = io.BytesIO(file_content)
```

What is `io.BytesIO`?

- Converts bytes (raw data) → file-like object
- PyPDF2 expects a file, not bytes
- Like wrapping bytes in a file costume

python

```
# Create PDF reader
pdf_reader = PyPDF2.PdfReader(pdf_file)
```

What is `PdfReader`?

- PyPDF2's main class for reading PDFs
- Can access pages, text, metadata

python

```
# Extract text from all pages
text_content = []
for page_num in range(len(pdf_reader.pages)):
    page = pdf_reader.pages[page_num]
    text = page.extract_text()
    text_content.append(text)
```

****Step-by-step:****

1. Loop through each page
2. Get page object
3. Extract text from that page
4. Add to list

Example:

Page 1 text: "COMPANY SECURITY POLICY 2024..."

Page 2 text: "Requirements: All employees..."

List: ["COMPANY SECURITY POLICY 2024...", "Requirements: All employees..."]

python

```
# Join all pages with double newline
full_text = "\n\n".join(text_content)
return full_text
```

What is "\n\n".join()?

- Combines list items with double newline between them
- Creates readable text with page breaks

Result:

"COMPANY SECURITY POLICY 2024..."

Requirements: All employees..."

Function 3: extract_dates_from_text()

This is the **AI DATE EXTRACTION** - most important function!

python

```
def extract_dates_from_text(text: str) -> Tuple[Optional[str], Optional[str]]:
    """
    Extract start and end dates from document text using AI pattern matching.
    """
    # Take first 500 characters (dates usually at the top)
    search_text = text[:500]
```

Why first 500 characters?

- Dates usually at top of policy documents
- Faster than searching entire document
- `text[:500]` = Python slice (first 500 chars)

python

```
# Define regex patterns for common date formats
patterns = {
    'effective_date': [
        r'Effective Date:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})',
        r'Effective:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})',
        ...
    ],
    ...
}
```

What are regex patterns?

Regex = Regular Expression = Pattern matching

Let's decode one pattern:

python

```
r'Effective Date:\s*([A-Za-z]+\s+\d{1,2},?\s+\d{4})'
```

****Breaking it down:****

`r'...'` = Raw string (treats `\` literally)

`Effective Date:` = Literal text to match

`\s*` = Zero or more whitespace

`(...)` = Capture group (save what's inside)

`[A-Za-z]+` = One or more letters (month name)

`\s+` = One or more spaces

`\d{1,2}` = 1 or 2 digits (day: 1-31)

`,?` = Optional comma

`\s+` = One or more spaces

`\d{4}` = Exactly 4 digits (year: 2024)

What it matches:

- "Effective Date: January 1, 2024" ✓
- "Effective Date:January 1,2024" ✓
- "Effective Date: Jan 15 2024" ✓

python

```
# Try to find effective date
for pattern in patterns['effective_date']:
    match = re.search(pattern, search_text, re.IGNORECASE)
    if match:
        date_str = match.group(1)
        parsed_date = dateparser.parse(date_str)
        if parsed_date:
            valid_from = parsed_date.strftime("%Y-%m-%d")
            break
```

Step-by-step:

1. Loop through patterns

- Try first pattern
- If no match, try second pattern
- Continue until match found

2. re.search(pattern, text, flags)

- Searches for pattern in text
- `re.IGNORECASE` = Match "Effective" or "effective"
- Returns match object or None

3. match.group(1)

- Gets the captured group (inside parentheses)
- Example: From "Effective Date: January 1, 2024"
- Captures: "January 1, 2024"

4. dateparser.parse(date_str)

- Smart date parsing
- Understands natural language

- Examples:

"January 1, 2024" → datetime(2024, 1, 1)

"Jan 15, 2024" → datetime(2024, 1, 15)

"2024-01-01" → datetime(2024, 1, 1)

"1/15/2024" → datetime(2024, 1, 15)

5. **strftime("%Y-%m-%d")**

a. Formats date to YYYY-MM-DD

b. Example: datetime(2024, 1, 1) → "2024-01-01"

6. **break**

a. Stop searching once we found a date

b. No need to try other patterns

python

```
# Fallback logic if dates not found
if valid_from and not valid_to:
    # If we have start but no end, assume end of year
    year = valid_from[:4]
    valid_to = f"{year}-12-31"
```

Fallback Strategy 1:

If we found start date but not end date:

Found: valid_from = "2024-01-01"

Missing: valid_to = None

Fallback:

Extract year: "2024-01-01"[:4] = "2024"

Assume end of year: valid_to = "2024-12-31"

python

```
if not valid_from and not valid_to:
    # Try to extract any 4-digit year
    year_match = re.search(r'(20\d{2})', search_text)
    if year_match:
        year = year_match.group(1)
```

```
valid_from = f"{year}-01-01"  
valid_to = f"{year}-12-31"
```

Fallback Strategy 2:

If we found NO dates at all:

Look for any 4-digit year starting with "20"

Pattern: (20\d{2})

Matches: 2020, 2021, 2022, ..., 2099

Example:

Text: "COMPANY POLICY 2024 Requirements..."

Extract: year = "2024"

Assume: valid_from = "2024-01-01"

valid_to = "2024-12-31"

Function 4: extract_metadata_from_pdf()

python

```
def extract_metadata_from_pdf(file_content: bytes) -> Dict:  
    """  
    Extract all metadata from PDF including text, dates, and additional info.  
    """  
  
    # Extract text  
    content = extract_text_from_pdf(file_content)  
  
    # Extract dates  
    valid_from, valid_to = extract_dates_from_text(content)
```

What this does:

1. Call extract_text_from_pdf() → Get full text
2. Call extract_dates_from_text() → Get dates

python

```
# Extract additional metadata
metadata = {}

# Try to extract policy version
version_match = re.search(r'Policy Version:\s*([\d\.]+)', content, re.IGNORECASE)
if version_match:
    metadata['policy_version'] = version_match.group(1)
```

What this does:

Look for "Policy Version: 2024.1" in document:

Pattern: Policy Version:\s*([\d\.]+)

Matches: "Policy Version: 2024.1"

"Policy Version:2.5"

"Policy version: 1.0"

Captures: "2024.1" (numbers and dots only)

Result: metadata['policy_version'] = "2024.1"

python

```
# Try to extract department
dept_match = re.search(r'Department:\s*([A-Za-z\s]+)', content, re.IGNORECASE)
if dept_match:
    metadata['department'] = dept_match.group(1).strip()
```

What this does:

Look for "Department: IT Security" in document:

Pattern: Department:\s*([A-Za-z\s]+)

Matches: "Department: IT Security"

"Department:Human Resources"

Captures: "IT Security" (letters and spaces)

Result: metadata['department'] = "IT Security"

python

```
# Mark extraction method
metadata['extraction_method'] = 'AI-automated'

return {
    'content': content,
    'valid_from': valid_from,
    'valid_to': valid_to,
    'metadata': metadata
}
```

Final return value:

python

```
{
  'content': "Full PDF text here...",
  'valid_from': "2024-01-01",
  'valid_to': "2024-12-31",
  'metadata': {
    'policy_version': "2024.1",
    'department': "IT Security",
    'extraction_method': "AI-automated"
  }
}
```

STEP 3: Add Models for PDF Upload

What we're doing:

- Adding data models for API requests/responses
- Defines structure of JSON data

Open app/models.py in VS Code

Scroll to the BOTTOM of the file

Add these lines:

python

```
# PDF Upload Models
class PDFUploadMetadata(BaseModel):
    policy_version: Optional[str] = None
    department: Optional[str] = None
    extraction_method: str

class PDFUploadResponse(BaseModel):
    status: str
    doc_id: str
    filename: str
    pages_extracted: int
    characters_extracted: int
    message: str
```

What these models do:

PDFUploadMetadata:

- Defines extra metadata we extract from PDFs
- Optional[str] = Can be None (not always present)
- str = Must be present

PDFUploadResponse:

- Defines API response structure
- Tells user what happened after upload

STEP 4: Create Smart PDF Upload Endpoints

What we're doing:

- Adding 2 new API endpoints:

- /api/smart-upload-pdf - Upload single PDF
- /api/smart-batch-upload-pdf - Upload multiple PDFs

Open app/api/routes.py in VS Code

Find the line with imports at the top

Add these imports:

python

```
from core.pdf_processor import (
    extract_text_from_pdf,
    validate_pdf,
    extract_metadata_from_pdf
)
from app.models import PDFUploadMetadata, PDFUploadResponse
```

Your imports section should now look like:

python

```
from fastapi import APIRouter, HTTPException, UploadFile, File, Form
from app.models import (
    QueryRequest,
    QueryResponse,
    DocumentResponse,
    PDFUploadMetadata, # NEW
    PDFUploadResponse # NEW
)
from typing import List, Optional
from core.retriever import get_retriever
from llm.gemini_client import get_gemini_client
from core.pdf_processor import ( # NEW
    extract_text_from_pdf,
    validate_pdf,
    extract_metadata_from_pdf
)
import json
```

Scroll down to find the section with routes

Add this NEW section BEFORE the /query endpoint:

python

```
# ===== 🤖 SMART PDF UPLOAD (AI AUTO-EXTRACTS DATES)
=====

@router.post("/smart-upload-pdf")
async def smart_upload_pdf(
    file: UploadFile = File(..., description="PDF file - dates will be auto-extracted")
):
    """
    🤖 SMART PDF UPLOAD - Automatically extracts dates from document!

    Just upload the PDF, AI will:
    - Extract all text
    - Find "Effective Date"
    - Find "Valid Through" date
    - Store with correct temporal validity

    No manual date entry needed!
    """
    try:
        # Validate file type
        if not file.filename.lower().endswith('.pdf'):
            raise HTTPException(status_code=400, detail="Only PDF files are allowed")

        # Read file
        file_content = await file.read()

        # Validate PDF
        if not validate_pdf(file_content):
            raise HTTPException(status_code=400, detail="Invalid PDF file")

        # 🤖 AI EXTRACTION - Get everything automatically!
        print(f"\n 🤖 Processing {file.filename} with AI extraction...")
        extracted_data = extract_metadata_from_pdf(file_content)
```

```

# Check if we got dates

if not extracted_data["valid_from"] or not extracted_data["valid_to"]:
    raise HTTPException(
        status_code=400,
        detail=f"Could not extract dates from PDF. Please ensure document contains
'Effective Date' and 'Valid Through' fields."
    )

# Add to system
retriever = get_retriever()
doc_id = retriever.add_document(
    content=extracted_data["content"],
    valid_from=extracted_data["valid_from"],
    valid_to=extracted_data["valid_to"],
    source=file.filename,
    metadata=extracted_data["metadata"]
)

return {
    "status": "success",
    "doc_id": doc_id,
    "filename": file.filename,
    "extracted_dates": {
        "valid_from": extracted_data["valid_from"],
        "valid_to": extracted_data["valid_to"]
    },
    "metadata": extracted_data["metadata"],
    "message": f"✅ Successfully auto-extracted dates and stored {file.filename}"
}

except HTTPException:
    raise
except Exception as e:
    import traceback
    traceback.print_exc()
    raise HTTPException(status_code=500, detail=f"Error: {str(e)}")

```

```

@router.post("/smart-batch-upload-pdf")
async def smart_batch_upload_pdfs(

    files: List[UploadFile] = File(..., description="Multiple PDFs - dates auto-extracted from
each")
):
    """
    🤖 SMART BATCH UPLOAD - Upload multiple PDFs, AI extracts dates from each!

    Just drag & drop all your policy PDFs!
    Each document's dates are extracted individually.
    """
    try:
        results = {
            "successful": 0,
            "failed": 0,
            "doc_ids": [],
            "documents": [],
            "errors": []
        }

        retriever = get_retriever()

        for file in files:
            try:
                print(f"\n 📄 Processing: {file.filename}")

                # Validate
                if not file.filename.lower().endswith('.pdf'):
                    results["failed"] += 1
                    results["errors"].append({
                        "filename": file.filename,
                        "error": "Not a PDF file"
                    })
                continue

```

```

# Read and validate
file_content = await file.read()
if not validate_pdf(file_content):
    results["failed"] += 1
    results["errors"].append({
        "filename": file.filename,

        "error": "Invalid PDF format"
    })
    continue

# 🤖 AI EXTRACTION
extracted_data = extract_metadata_from_pdf(file_content)

# Check dates
if not extracted_data["valid_from"] or not extracted_data["valid_to"]:
    results["failed"] += 1
    results["errors"].append({
        "filename": file.filename,
        "error": "Could not extract dates from document"
    })
    continue

# Add to system
doc_id = retriever.add_document(
    content=extracted_data["content"],
    valid_from=extracted_data["valid_from"],
    valid_to=extracted_data["valid_to"],
    source=file.filename,
    metadata=extracted_data["metadata"]
)

results["successful"] += 1
results["doc_ids"].append(doc_id)
results["documents"].append({
    "filename": file.filename,
    "doc_id": doc_id,
    "valid_from": extracted_data["valid_from"],

```

```

        "valid_to": extracted_data["valid_to"],
        "metadata": extracted_data["metadata"]
    })

    print(f"✅ {file.filename}: {extracted_data['valid_from']} to
{extracted_data['valid_to']}")

except Exception as e:

    results["failed"] += 1
    results["errors"].append({
        "filename": file.filename,
        "error": str(e)
    })
    print(f"❌ Error with {file.filename}: {e}")

return {
    "status": "completed",
    "total_files": len(files),
    "successful": results["successful"],
    "failed": results["failed"],
    "documents": results["documents"],
    "errors": results["errors"]
}

except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

```

Code Explanation - smart_upload_pdf endpoint:

python

```

@router.post("/smart-upload-pdf")
async def smart_upload_pdf(
    file: UploadFile = File(..., description="PDF file - dates will be auto-extracted")
):

```

What is @router.post()?

- Creates a POST endpoint at /api/smart-upload-pdf
- Users send files to this URL

What is UploadFile?

- FastAPI type for file uploads
- Handles multipart/form-data (how files are uploaded)

What is File(...)?

- Tells FastAPI this is a required file upload
- ... means required (cannot be empty)

python

```
# Validate file type
if not file.filename.lower().endswith('.pdf'):
    raise HTTPException(status_code=400, detail="Only PDF files are allowed")
```

What this does:

- Checks if filename ends with ".pdf"
- .lower() = convert to lowercase ("Policy.PDF" → "policy.pdf")
- If not PDF, return error 400 (Bad Request)

python

```
# Read file
file_content = await file.read()
```

What is await?

- FastAPI is asynchronous
- await waits for file to be fully read
- Returns file as bytes

python

```
# Validate PDF
if not validate_pdf(file_content):
    raise HTTPException(status_code=400, detail="Invalid PDF file")
```

What this does:

- Calls our validate_pdf() function
- Checks if file starts with "%PDF"
- If not valid, return error

python

```
# 🤖 AI EXTRACTION - Get everything automatically!
print(f"\n🤖 Processing {file.filename} with AI extraction...")
extracted_data = extract_metadata_from_pdf(file_content)
```

What happens here:

1. Calls our AI extraction function
2. Returns dictionary:

python

```
{
    'content': "Full PDF text...",
    'valid_from': "2024-01-01",
    'valid_to': "2024-12-31",
    'metadata': {...}
}
```

python

```
# Check if we got dates
if not extracted_data["valid_from"] or not extracted_data["valid_to"]:
    raise HTTPException(
        status_code=400,
        detail="Could not extract dates from PDF. Please ensure document contains 'Effective Date' and 'Valid Through' fields."
    )
```

What this does:

If AI couldn't find dates:

- Return error 400
- Tell user what went wrong
- User knows to check PDF format

python

```
# Add to system
retriever = get_retriever()
doc_id = retriever.add_document(
    content=extracted_data["content"],
    valid_from=extracted_data["valid_from"],
    valid_to=extracted_data["valid_to"],
    source=file.filename,
    metadata=extracted_data["metadata"]
)
```

What happens here:

1. Get retriever instance
2. Call add_document() with extracted data
3. This:
 - a. Generates embedding
 - b. Stores in FAISS
 - c. Stores in PostgreSQL
 - d. Returns doc_id

python

```
return {
    "status": "success",
    "doc_id": doc_id,
    "filename": file.filename,
    "extracted_dates": {
        "valid_from": extracted_data["valid_from"],
        "valid_to": extracted_data["valid_to"]
    }
}
```

```
    },  
    "metadata": extracted_data["metadata"],  
    "message": f"✅ Successfully auto-extracted dates and stored {file.filename}"  
}
```

What gets returned:

JSON response showing:

- Success status
- Document ID
- What dates were extracted
- Any extra metadata found

Code Explanation - smart_batch_upload_pdfs endpoint:

This endpoint is similar but processes multiple PDFs:

python

```
for file in files:  
    try:  
        # Process each file individually  
        ...  
        results["successful"] += 1  
    except Exception as e:  
        # If one file fails, continue with others  
        results["failed"] += 1  
        results["errors"].append(...)
```

Key differences:

1. **Accepts List[UploadFile]** - Multiple files
2. **Processes each individually** - One failure doesn't stop others
3. **Tracks statistics** - Successful vs Failed counts
4. **Returns detailed results** - Per-file success/failure

Save the file: Cmd+S or Ctrl+S

STEP 5: Test Smart PDF Upload

Start the server:

bash

```
# Make sure you're in backend directory with venv activated
python -m app.main
\ \ \

**Expected output:**
\ \ \

📦 Loading embedding model: sentence-transformers/all-MiniLM-L6-v2
✅ Model loaded! Embedding dimension: 384
📁 VectorStore initialized
✅ Loaded index with X vectors
📋 MetadataStore initialized (PostgreSQL)
🧠 TimeAwareRetriever initialized
INFO: Uvicorn running on http://0.0.0.0:8000
```

Open browser: <http://localhost:8000/docs>

Find: POST /api/smart-upload-pdf

Click: "Try it out"

Click: "Choose File"

Select: Any PDF with dates (like password_policy_2025.pdf)

Click: "Execute"

Expected Response (200):

json

```
{
  "status": "success",
  "doc_id": "uuid-here",
  "filename": "password_policy_2025.pdf",
  "extracted_dates": {
    "valid_from": "2025-01-01",
    "valid_to": "2025-12-31"
  },
  "metadata": {
    "policy_version": "2025.1",
    "department": null,
    "extraction_method": "AI-automated"
  },
  "message": "✅ Successfully auto-extracted dates and stored password_policy_2025.pdf"
}
```

****Check terminal output:****

```
\ \ \

🖨️ Processing password_policy_2025.pdf with AI extraction...
✅ password_policy_2025.pdf: 2025-01-01 to 2025-12-31
✅ Added document to PostgreSQL: 07a07dcd...
```

Test Batch Upload:

Find: POST /api/smart-batch-upload-pdf

Click: "Try it out"

Select multiple PDFs (5 files):

- vpn_policy_2022.pdf
- remote_work_2023.pdf
- zerotrust_policy_2024.pdf
- meeting_policy_2024.pdf

- password_policy_2025.pdf

Click: "Execute"

Expected Response:

json

```
{
  "status": "completed",
  "total_files": 5,
  "successful": 5,
  "failed": 0,
  "documents": [
    {
      "filename": "vpn_policy_2022.pdf",
      "doc_id": "uuid-1",
      "valid_from": "2022-01-01",
      "valid_to": "2023-12-31",
      "metadata": {
        "policy_version": "2022.1",
        "extraction_method": "AI-automated"
      }
    },
    // ... 4 more documents
  ],
  "errors": []
}
```

Verify in Database:

bash

psql timeaware_rag

sql

```
SELECT doc_id, source, valid_from, valid_to
FROM documents
```

ORDER BY created_at DESC

LIMIT 5;

```

**\*\*Expected output:\*\***

```

doc_id	source	valid_from	valid_to
-----+-----+-----+-----			
uuid-5	password_policy_2025.pdf	2025-01-01	2025-12-31
uuid-4	meeting_policy_2024.pdf	2024-01-01	2024-12-31
uuid-3	zerotrust_policy_2024..	2024-01-01	2025-12-31
uuid-2	remote_work_2023.pdf	2023-01-01	2023-12-31
uuid-1	vpn_policy_2022.pdf	2022-01-01	2023-12-31

Exit:

sql

\q

```

---

### What We Accomplished - Feature 1

**\*\*Before Feature 1:\*\***

```

- X Manual date entry for each PDF
- X Time-consuming bulk uploads
- X Human errors in date typing
- X No metadata extraction

```

**\*\*After Feature 1:\*\***

```

- ✓ AI automatically extracts dates
- ✓ Batch process 100s of PDFs in minutes
- ✓ Zero manual typing

- ✓ Extracts policy versions and departments
- ✓ 40x faster document uploads

Skills Demonstrated:

- PDF processing with PyPDF2
- Regex pattern matching
- Natural language date parsing
- FastAPI file uploads
- Error handling and validation